CNS Lab External Examination – Java

Java Programs and Part (b) Theory Answers

Prepared from the supplied CNS Lab question-paper answers. Java code is kept in monospaced blocks for easier copying.

CNS Lab External Examination – Java Programs and Answers

Based on the uploaded G. Narayanamma Institute of Technology and Science CNS Lab question paper (14 questions).

Note: Programs are educational implementations. Classical ciphers and DES/RC4 are obsolete and should not be used to protect real data. For practical use, use modern authenticated cryptography such as AES-GCM.

Question 1(a): Caesar Cipher

Aim: To implement encryption and decryption using Caesar Cipher.

Algorithm: Shift each alphabetic character by a key k during encryption; shift backward by k during decryption. Preserve case and leave non-letters unchanged.

Java program:

```
import java.util.Scanner;

public class CaesarCipher {
    static String transform(String text, int key) {
        StringBuilder out = new StringBuilder();
        key = ((key % 26) + 26) % 26;
        for (char c : text.toCharArray()) {
            if (c >= 'A' && c <= 'Z')
                out.append((char) ('A' + (c - 'A' + key) % 26));
            else if (c >= 'a' && c <= 'z')
                out.append((char) ('a' + (c - 'a' + key) % 26));
            else out.append(c);
        }
        return out.toString();
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String text = sc.nextLine();
        System.out.print("Enter key (integer): ");
        int key = sc.nextInt();
        String encrypted = transform(text, key);
        System.out.println("Encrypted: " + encrypted);
        System.out.println("Decrypted: " + transform(encrypted, -key));
        sc.close();
    }
}
```

Sample output: Enter text: HELLO WORLD | Enter key (integer): 3 | Encrypted: KHOOR ZRUOG | Decrypted: HELLO WORLD

Question 1(b): Windows Firewall

Aim: To study firewall features and configure Windows Firewall.

Features: filters inbound/outbound traffic; supports domain/private/public profiles; allows or blocks apps and ports; logs selected events; helps reduce unauthorized network access.

Steps: Open Start → search “Windows Security” → Firewall & network protection. Check Domain, Private, and Public profiles. Choose a profile and ensure Microsoft Defender Firewall is On. Use “Allow an app through firewall” only for trusted apps. For advanced rules, open “Windows Defender Firewall with Advanced Security” and create an inbound/outbound rule only when required. Do not disable the firewall for routine testing.

Question 2(a): Playfair Cipher

Aim: Implement Playfair encryption and decryption. Convention: I/J share one cell; repeated letters in a pair are separated with X; append X to an odd-length final pair. This teaching implementation handles alphabetic input.

Java program:

```
import java.util.*;

public class PlayfairCipher {
    static char[][] square = new char[5][5];
    static Map<Character, int[]> pos = new HashMap<>();

    static void build(String key) {
        StringBuilder s = new StringBuilder();
        key = (key + "ABCDEFGHIJKLMNOPQRSTUVWXYZ").toUpperCase().replace('J','I');
        for (char c : key.toCharArray())
            if (c >= 'A' && c <= 'Z' && s.indexOf("" + c) < 0) s.append(c);
        int k = 0;
        for (int r=0; r<5; r++) for (int c=0; c<5; c++) {
            square[r][c] = s.charAt(k++);
            pos.put(square[r][c], new int[]{r,c});
        }
    }
    static String prepare(String text) {
        String t = text.toUpperCase().replaceAll("[^A-Z]", "").replace('J','I');
        StringBuilder p = new StringBuilder();
        for (int i=0; i<t.length();) {
            char a=t.charAt(i++);
            char b = i<t.length()?t.charAt(i):'X';
            if (i<t.length() && a!=b) i++;
            else b='X';
            p.append(a).append(b);
        }
        return p.toString();
    }
    static String crypt(String text, boolean decrypt) {
        String t = decrypt ? text.toUpperCase().replaceAll("[^A-Z]", "").replace('J','I') : prepare(text);
        if (t.length()%2!=0) t += "X";
        StringBuilder out = new StringBuilder();
        int d=decrypt?-1:1;
        for(int i=0;i<t.length();i+=2){
            int[] a=pos.get(t.charAt(i)), b=pos.get(t.charAt(i+1));
            if(a[0]==b[0]){
                out.append(square[a[0]][(a[1]+d+5)%5]);
                out.append(square[b[0]][(b[1]+d+5)%5]);
            } else if(a[1]==b[1]){
                out.append(square[(a[0]+d+5)%5][a[1]]);
                out.append(square[(b[0]+d+5)%5][b[1]]);
            } else {
                out.append(square[a[0]][b[1]]).append(square[b[0]][a[1]]);
            }
        }
        return out.toString();
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter key: "); build(sc.nextLine());
        System.out.print("Enter plaintext: "); String p=sc.nextLine();
        String e=crypt(p,false);
        System.out.println("Encrypted: "+e);
        System.out.println("Decrypted (prepared text): "+crypt(e,true));
        sc.close();
    }
}
```

Sample output (key MONARCHY, plaintext INSTRUMENTS): Encrypted output depends on standard Playfair pairing rules; run the program to display the exact result. Decryption returns prepared text, so inserted X characters may remain.

Question 2(b): Browser Security (Google Chrome)

1. Update Chrome from Settings → About Chrome. 2. Turn on Safe Browsing (Enhanced or Standard) under Privacy and security → Security. 3. Use a strong, unique password and enable multi-factor authentication on important accounts. 4. Review site permissions for camera, microphone, location and notifications. 5. Remove unknown extensions. 6. Avoid suspicious downloads and HTTPS certificate warnings. 7. Clear browsing data on shared devices and use a separate profile for shared computers.

Question 3(a): Hill Cipher (2×2)

Aim: Encrypt/decrypt letters in blocks of two using a 2×2 key matrix modulo 26. This implementation accepts only a key matrix whose determinant has a modular inverse modulo 26.

Java program:

```
import java.util.*;
public class HillCipher {
    static int mod(int x){ return (x%26+26)%26; }
    static int inv(int a){
        for(int i=1;i<26;i++) if(mod(a*i)==1) return i;
        return -1;
    }
    static String clean(String s){ return s.toUpperCase().replaceAll("[^A-Z]",""); }
    static String crypt(String text,int[][] k){
        String t=clean(text); if(t.length()%2==1)t+="X";
        StringBuilder o=new StringBuilder();
        for(int i=0;i<t.length();i+=2){
            int a=t.charAt(i)-'A', b=t.charAt(i+1)-'A';
            o.append((char)('A'+mod(k[0][0]*a+k[0][1]*b)));
            o.append((char)('A'+mod(k[1][0]*a+k[1][1]*b)));
        } return o.toString();
    }
    static int[][] inverse(int[][] k){
        int det=mod(k[0][0]*k[1][1]-k[0][1]*k[1][0]), di=inv(det);
        if(di<0) throw new IllegalArgumentException("Key matrix is not invertible modulo 26");
        return new int[][]{{mod(di*k[1][1]),mod(-di*k[0][1])},
                           {mod(-di*k[1][0]),mod(di*k[0][0])}};
    }
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in); int[][] k=new int[2][2];
        System.out.println("Enter 2x2 key matrix values (0-25):");
        for(int r=0;r<2;r++)for(int c=0;c<2;c++)k[r][c]=sc.nextInt();
        sc.nextLine(); System.out.print("Enter plaintext: "); String p=sc.nextLine();
        String e=crypt(p,k); System.out.println("Encrypted: "+e);
        System.out.println("Decrypted: "+crypt(e,inverse(k)));
        sc.close();
    }
}
```

Example key matrix [[3,3],[2,5]], plaintext HELP. The program prints ciphertext and recovers HEL P with padding if needed; exact output is generated by running the program.

Question 3(b): Website/Web Application Vulnerabilities

Common categories: SQL injection; cross-site scripting (XSS); broken access control/IDOR; authentication and session weaknesses; security misconfiguration; vulnerable/outdated components; insecure file upload; path traversal; sensitive-data exposure; cross-site request forgery (CSRF). Prevention: parameterized SQL queries, output encoding, authorization checks on every request, MFA and secure sessions, patching, least privilege, safe upload validation, HTTPS, security headers, and authorized testing only.

Question 4(a): Rail Fence Cipher

Java program:

```
import java.util.*;
public class RailFence {
    static String encrypt(String s,int rails){
        if(rails<2 || rails>=s.length()) return s;
        StringBuilder[] rows=new StringBuilder[rails];
        for(int i=0;i<rails;i++)rows[i]=new StringBuilder();
        int r=0,dir=1;
        for(char c:s.toCharArray()){
            rows[r].append(c);
            if(r==0)dir=1; else if(r==rails-1)dir=-1;
            r+=dir;
        }
        StringBuilder out=new StringBuilder();
        for(StringBuilder row:rows)out.append(row);
        return out.toString();
    }
    static String decrypt(String s,int rails){
        if(rails<2 || rails>=s.length()) return s;
        int n=s.length(); int[] rowAt=new int[n]; int r=0,dir=1;
        for(int i=0;i<n;i++){rowAt[i]=r;if(r==0)dir=1;else if(r==rails-1)dir=-1;r+=dir;}
        int[] counts=new int[rails];for(int x:rowAt)counts[x]++;
        String[] rows=new String[rails];int p=0;
        for(int i=0;i<rails;i++){rows[i]=s.substring(p,p+counts[i]);p+=counts[i];}
        int[] idx=new int[rails];StringBuilder out=new StringBuilder();
        for(int x:rowAt)out.append(rows[x].charAt(idx[x]++));
        return out.toString();
    }
    public static void main(String[] a){
        Scanner sc=new Scanner(System.in);System.out.print("Enter text (no spaces for this demo): ");
        String p=sc.nextLine().replaceAll("\\s+","");
        System.out.print("Enter rails: ");int n=sc.nextInt();
        String e=encrypt(p,n);System.out.println("Encrypted: "+e);
        System.out.println("Decrypted: "+decrypt(e,n));sc.close();
    }
}
```

Sample output: plaintext WEAREDISCOVERED, rails 3 → encrypted WECRERDSOEEAIVD; decrypted WEAREDISCOVERED.

Question 4(b): E-mail Application Security Vulnerabilities

Risks: phishing and spoofing; weak/reused passwords; account takeover; malicious attachments/links; insecure public Wi-Fi; misconfigured forwarding rules; data leakage; outdated mail clients; insufficient encryption. Mitigations: MFA, unique passwords, phishing awareness, verify sender and URLs, scan attachments, update clients, use TLS, review forwarding/recovery settings, and use end-to-end encryption for sensitive content where appropriate.

Question 5(a): Row & Column Transformation (simple transposition)

Convention: write text row-wise into a rectangular grid, pad with X, then read column-wise. This is a simple educational transposition, not modern encryption.

Java program:

```
import java.util.*;
public class RowColumn {
    static String encrypt(String s,int cols){
        s=s.toUpperCase().replaceAll("\\s+","");
        int rows=(s.length()+cols-1)/cols;
        char[][] g=new char[rows][cols];int k=0;
        for(int r=0;r<rows;r++)for(int c=0;c<cols;c++)g[r][c]=k<s.length()?s.charAt(k++):'X';
        StringBuilder o=new StringBuilder();
        for(int c=0;c<cols;c++)for(int r=0;r<rows;r++)o.append(g[r][c]);
        return o.toString();
    }
    static String decrypt(String s,int cols){
        int rows=(s.length()+cols-1)/cols;
        char[][] g=new char[rows][cols];int k=0;
        for(int c=0;c<cols;c++)for(int r=0;r<rows;r++)g[r][c]=s.charAt(k++);
        StringBuilder o=new StringBuilder();
        for(int r=0;r<rows;r++)for(int c=0;c<cols;c++)o.append(g[r][c]);
        while(o.length()>0&&o.charAt(o.length()-1)=='X')o.setLength(o.length()-1);
        return o.toString();
    }
    public static void main(String[] a){
        Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String p=sc.nextLine();
        System.out.print("Number of columns: ");int c=sc.nextInt();
        if(c<1){System.out.println("Columns must be positive.");return;}
        String e=encrypt(p,c);System.out.println("Encrypted: "+e);
        System.out.println("Decrypted: "+decrypt(e,c));sc.close();
    }
}
```

Example: plaintext HELLOWORLD, 4 columns → grid rows HEL L / OWOR / LDXX (grid padded row-wise); ciphertext is read column-wise. Run program for exact output.

Question 5(b): Windows Firewall

Same procedure and features as Q1(b). Verify all relevant profiles are enabled; add only necessary application/port rules and keep default inbound blocking unless policy requires otherwise.

Question 6(a): DES Algorithm

DES is a legacy 56-bit-key block cipher and is insecure for modern protection. This Java example uses the built-in DES/ECB/PKCS5Padding provider for lab demonstration only.

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;
public class DESDemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String text=sc.nextLine();
KeyGenerator kg=KeyGenerator.getInstance("DES");kg.init(56);SecretKey key=kg.generateKey();
Cipher c=Cipher.getInstance("DES/ECB/PKCS5Padding");
c.init(Cipher.ENCRYPT_MODE,key);
byte[] encrypted=c.doFinal(text.getBytes(java.nio.charset.StandardCharsets.UTF_8));
System.out.println("Encrypted (Base64): "+Base64.getEncoder().encodeToString(encrypted));
c.init(Cipher.DECRYPT_MODE,key);
System.out.println("Decrypted: "+new String(c.doFinal(encrypted),java.nio.charset.StandardCharsets.UTF_8));
sc.close();
}
}
```

Output: ciphertext varies each run because a new key is generated; decrypted output equals the entered text.

Question 6(b): Browser Security

See Q2(b), Google Chrome security steps.

Question 7(a): AES Algorithm

AES is a modern symmetric block cipher. This lab demo uses AES/GCM/NoPadding, which provides confidentiality and integrity. Keep the generated key and nonce for decryption; never reuse a nonce with the same key.

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.Scanner;
public class AESDemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String text=sc.nextLine();
KeyGenerator kg=KeyGenerator.getInstance("AES");kg.init(128);SecretKey key=kg.generateKey();
byte[] nonce=new byte[12];new SecureRandom().nextBytes(nonce);
Cipher c=Cipher.getInstance("AES/GCM/NoPadding");
c.init(Cipher.ENCRYPT_MODE,key,new GCMParameterSpec(128,nonce));
byte[] ciphertext=c.doFinal(text.getBytes(java.nio.charset.StandardCharsets.UTF_8));
System.out.println("Nonce: "+Base64.getEncoder().encodeToString(nonce));
System.out.println("Ciphertext (Base64): "+Base64.getEncoder().encodeToString(ciphertext));
c.init(Cipher.DECRYPT_MODE,key,new GCMParameterSpec(128,nonce));
System.out.println("Decrypted: "+new String(c.doFinal(ciphertext),java.nio.charset.StandardCharsets.UTF_8));
sc.close();
}
}
```

Output: nonce/ciphertext vary each run; decrypted output equals the entered text.

Question 7(b): Website Vulnerabilities

See Q3(b): SQL injection, XSS, broken access control, authentication/session issues, misconfiguration, outdated components, insecure uploads, path traversal, CSRF and sensitive-data exposure. Test only systems for which you have permission.

Question 8(a): ECB Block Mode

ECB encrypts each block independently and leaks repeated-block patterns. It is included only to demonstrate the mode; do not use ECB for real data.

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;
public class ECBDemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String p=sc.nextLine();
KeyGenerator kg=KeyGenerator.getInstance("AES");kg.init(128);SecretKey key=kg.generateKey();
Cipher c=Cipher.getInstance("AES/ECB/PKCS5Padding");
c.init(Cipher.ENCRYPT_MODE,key);
byte[] e=c.doFinal(p.getBytes(java.nio.charset.StandardCharsets.UTF_8));
System.out.println("Encrypted (Base64): "+Base64.getEncoder().encodeToString(e));
c.init(Cipher.DECRYPT_MODE,key);
System.out.println("Decrypted: "+new String(c.doFinal(e),java.nio.charset.StandardCharsets.UTF_8));
sc.close();
}
}
```

Output: ciphertext varies with the randomly generated key; decrypted text equals the input.

Question 8(b): E-mail Security

See Q4(b): phishing, spoofing, weak passwords, account takeover, malicious attachments, unsafe Wi-Fi, forwarding-rule abuse and data leakage. Use MFA, unique passwords, updates, TLS and careful link/attachment handling.

Question 9(a): CBC Block Mode

CBC combines each plaintext block with the previous ciphertext block and uses a random IV. This demonstration uses AES/CBC/PKCS5Padding. CBC does not authenticate ciphertext; use authenticated encryption such as GCM in real systems.

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import java.security.SecureRandom;
import java.util.Base64;
import java.util.Scanner;
public class CBCDemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String p=sc.nextLine();
KeyGenerator kg=KeyGenerator.getInstance("AES");kg.init(128);SecretKey key=kg.generateKey();
byte[] iv=new byte[16];new SecureRandom().nextBytes(iv);IvParameterSpec ivSpec=new IvParameterSpec(iv);
Cipher c=Cipher.getInstance("AES/CBC/PKCS5Padding");
c.init(Cipher.ENCRYPT_MODE,key,ivSpec);
byte[] e=c.doFinal(p.getBytes(java.nio.charset.StandardCharsets.UTF_8));
System.out.println("IV: "+Base64.getEncoder().encodeToString(iv));
System.out.println("Encrypted (Base64): "+Base64.getEncoder().encodeToString(e));
c.init(Cipher.DECRYPT_MODE,key,ivSpec);
System.out.println("Decrypted: "+new String(c.doFinal(e),java.nio.charset.StandardCharsets.UTF_8));
sc.close();
}
}
```

Output: IV/ciphertext vary each run; decrypted output equals the entered text.

Question 9(b): Windows Firewall

See Q1(b) and Q5(b). Keep firewall enabled on Domain, Private and Public profiles; create only necessary rules.

Question 10(a): RC4 Algorithm

RC4 is obsolete and insecure. This small implementation demonstrates the RC4 keystream XOR process only for classroom study.

```
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Scanner;
public class RC4Demo {
static byte[] rc4(byte[] input, byte[] key) {
int[] s=new int[256];for(int i=0;i<256;i++)s[i]=i;
int j=0;
for(int i=0;i<256;i++){j=(j+s[i]+(key[i%key.length]&255))&255;int t=s[i];s[i]=s[j];s[j]=t;}
byte[] out=new byte[input.length];int i=0;j=0;
for(int n=0;n<input.length;n++){
i=(i+1)&255;j=(j+s[i])&255;int t=s[i];s[i]=s[j];s[j]=t;
int k=s[(s[i]+s[j])&255];out[n]=(byte)(input[n]^k);
}return out;
}
public static void main(String[] a){
Scanner sc=new Scanner(System.in);System.out.print("Enter key: ");byte[]
key=sc.nextLine().getBytes(StandardCharsets.UTF_8);
if(key.length==0){System.out.println("Key cannot be empty.");return;}
System.out.print("Enter plaintext: ");byte[] p=sc.nextLine().getBytes(StandardCharsets.UTF_8);
```

```
byte[] e=rc4(p,key);System.out.println("Encrypted (Base64): "+Base64.getEncoder().encodeToString(e));
System.out.println("Decrypted: "+new String(rc4(e,key),StandardCharsets.UTF_8));sc.close();
}
}
```

Sample output: encrypted Base64 varies with the input/key; decryption returns the original text.

Question 10(b): Browser Security

See Q2(b), Google Chrome security steps.

Question 11(a): RSA Algorithm

This demonstration uses Java RSA with OAEP padding, appropriate for encrypting short messages. For long files, use hybrid encryption.

```
import java.security.*;
import javax.crypto.Cipher;
import java.util.Base64;
import java.util.Scanner;
public class RSADemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter short text: ");String p=sc.nextLine();
KeyPairGenerator kg=KeyPairGenerator.getInstance("RSA");kg.initialize(2048);
KeyPair kp=kg.generateKeyPair();
Cipher c=Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
c.init(Cipher.ENCRYPT_MODE,kp.getPublic());
byte[] e=c.doFinal(p.getBytes(java.nio.charset.StandardCharsets.UTF_8));
System.out.println("Encrypted (Base64): "+Base64.getEncoder().encodeToString(e));
c.init(Cipher.DECRYPT_MODE,kp.getPrivate());
System.out.println("Decrypted: "+new String(c.doFinal(e),java.nio.charset.StandardCharsets.UTF_8));
sc.close();
}
}
```

Output: encrypted value varies each run; decrypted text equals the input.

Question 11(b): Website Vulnerabilities

See Q3(b). Prevention includes prepared statements, output encoding, server-side authorization, secure session handling, patching, validation and authorized security testing.

Question 12(a): Diffie–Hellman Key Exchange

Toy integer example for learning only; production use requires vetted cryptographic libraries and authenticated key exchange to prevent man-in-the-middle attacks.

```
import java.math.BigInteger;
import java.security.SecureRandom;
import java.util.Scanner;
public class DiffieHellmanDemo {
public static void main(String[] args){
Scanner sc=new Scanner(System.in);
System.out.print("Enter prime p (small demo prime): ");BigInteger p=sc.nextBigInteger();
System.out.print("Enter generator g: ");BigInteger g=sc.nextBigInteger();
SecureRandom random=new SecureRandom();
BigInteger a=new BigInteger(16,random).add(BigInteger.TWO);
BigInteger b=new BigInteger(16,random).add(BigInteger.TWO);
BigInteger A=g.modPow(a,p), B=g.modPow(b,p);
BigInteger sharedA=B.modPow(a,p), sharedB=A.modPow(b,p);
System.out.println("Public value A: "+A);
System.out.println("Public value B: "+B);
System.out.println("Shared key at Alice: "+sharedA);
```

```
System.out.println("Shared key at Bob: "+sharedB);
System.out.println("Keys match: "+sharedA.equals(sharedB));sc.close();
}
}
```

Sample: choose p=23 and g=5. Public/shared values vary because private exponents are random; “Keys match: true” should appear.

Question 12(b): Windows Firewall

See Q1(b): enable firewall for applicable profiles, review allowed apps, and add narrowly scoped rules only when needed.

Question 13(a): SHA-512 Message Digest

SHA-512 is a one-way cryptographic hash function. The same text yields the same 512-bit (128 hexadecimal-character) digest.

```
import java.security.MessageDigest;
import java.nio.charset.StandardCharsets;
import java.util.HexFormat;
import java.util.Scanner;
public class SHA512Demo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String text=sc.nextLine();
byte[] digest=MessageDigest.getInstance("SHA-512").digest(text.getBytes(StandardCharsets.UTF_8));
System.out.println("SHA-512: "+HexFormat.of().formatHex(digest));sc.close();
}
}
```

Sample output for text “abc”: ddaf35a193617abacc417349ae20413112e6fa4e89a97ea20a9eeee64b55d39a2192992a27
4fc1a836ba3c23a3feebbd454d4423643ce80e2a9ac94fa54ca49f

Question 13(b): Browser Security

See Q2(b). Keep Chrome updated, enable Safe Browsing, remove untrusted extensions, review site permissions and avoid suspicious downloads.

Question 14(a): Digital Signature on Text

Create an RSA key pair, sign text with SHA256withRSA using the private key, and verify using the public key.

```
import java.security.*;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Scanner;
public class DigitalSignatureDemo {
public static void main(String[] args) throws Exception {
Scanner sc=new Scanner(System.in);System.out.print("Enter text: ");String text=sc.nextLine();
KeyPairGenerator kg=KeyPairGenerator.getInstance("RSA");kg.initialize(2048);
KeyPair kp=kg.generateKeyPair();
byte[] data=text.getBytes(StandardCharsets.UTF_8);
Signature signer=Signature.getInstance("SHA256withRSA");
signer.initSign(kp.getPrivate());signer.update(data);byte[] sig=signer.sign();
System.out.println("Signature (Base64): "+Base64.getEncoder().encodeToString(sig));
Signature verifier=Signature.getInstance("SHA256withRSA");
verifier.initVerify(kp.getPublic());verifier.update(data);
System.out.println("Signature verified: "+verifier.verify(sig));sc.close();
}
}
```

Output: signature varies each run; “Signature verified: true”.

Question 14(b): Website Vulnerabilities

See Q3(b): common vulnerabilities include injection, XSS, broken access control, weak authentication, misconfiguration, outdated components, insecure uploads, path traversal and CSRF. Use secure coding and authorized testing.

Quick compilation note

Save each public class in a file with the same name (for example, CaesarCipher.java). Compile with javac CaesarCipher.java and run with java CaesarCipher. Use a Java version supporting HexFormat (Java 17+) for SHA512Demo; on older Java, convert bytes to hex manually.